

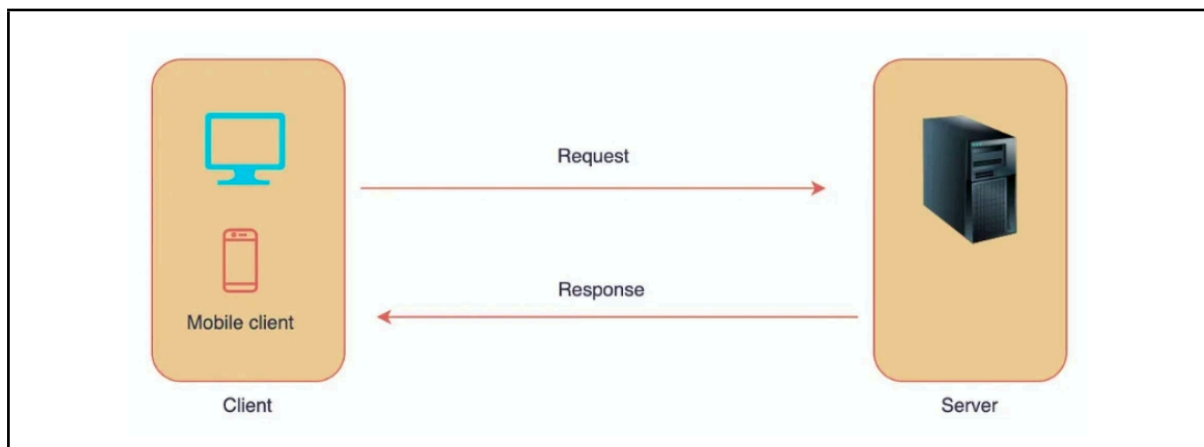
Section 7: Storage in System Design

Storage & Databases

In this section, we will explore how modern systems store, manage, and scale data, the foundation of every reliable and scalable application. In this first lecture we will cover the introduction to storage and system design and CAP theorem. We will begin by understanding storage fundamentals and the critical CAP theorem trade-offs that shape real-world distributed systems. So let's get started.

Why Storage Matters in System Design

Every scalable system is ultimately built around data. Users create it, services process it, and the business depends on preserving it over time. The moment data needs to survive beyond single request or server restart, storage becomes a foundational architectural concern rather than just an implementation detail.

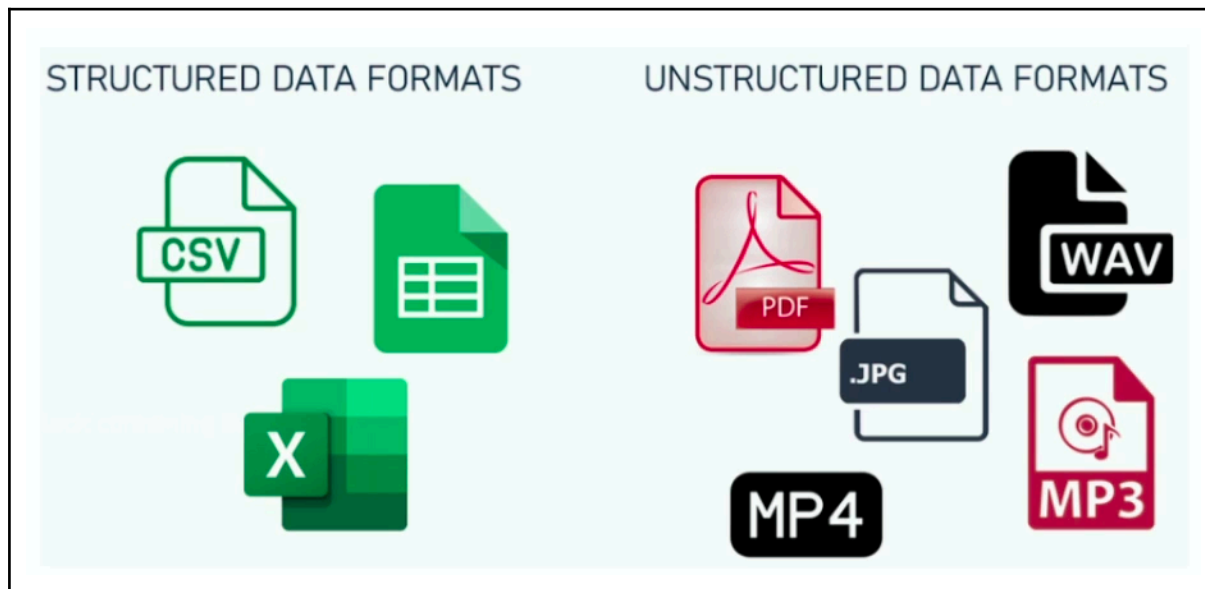


What makes storage especially important in system design is that it influences multiple dimensions of the system at once. The storage choices that you use affect response time, system reliability, operational cost, scalability limits, and even the user experience. A fast application backed by slow storage will eventually become a bottleneck, while unreliable storage can turn a minor failure into permanent data loss. Think about the features users interact with every day - profiles, preferences, shopping cards, search history, recommendations, analytics, and audit logs. All of these depend on persistent storage that can reliably retain and serve data over long periods of time.

As we move through this section, you will see that storage is not just where data lives. It is about designing systems that can store growing volumes of data efficiently, retrieve it quickly, and remain reliable as the systems scale.

Structured Vs. Unstructured Data

To make good storage decisions, architects first need to understand what kind of data they are dealing with. Not all data behaves the same way, and that is why we generally divide it into two broad categories - structured and unstructured data.

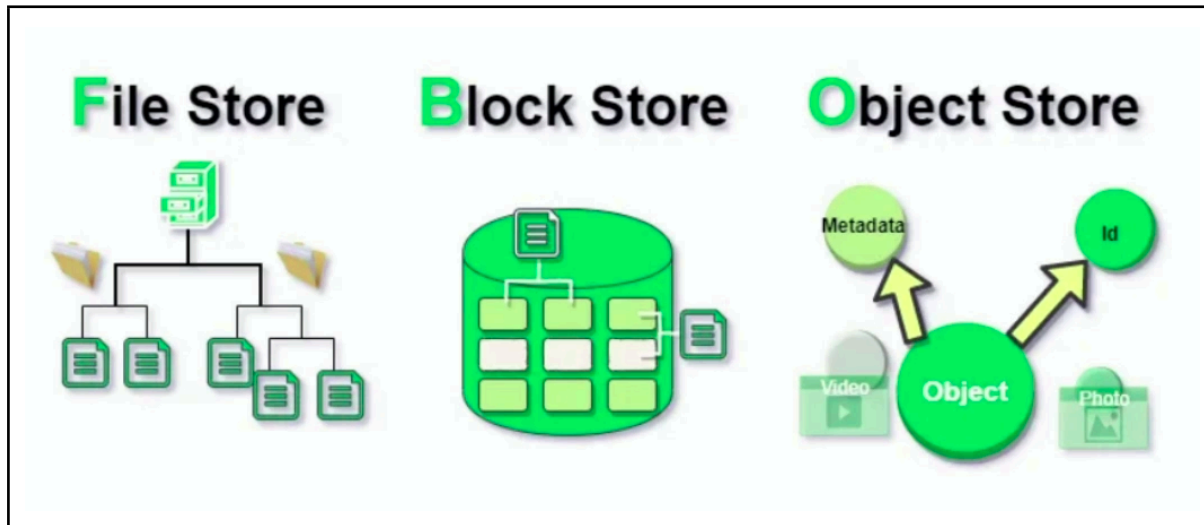


Structured data is highly organized and follows a predefined schema. Think about user accounts, orders, payments, or inventory records. Every row follows the same format, which makes querying, filtering, and joining data extremely efficient. These predictability characteristics of structured data is the reason why relational databases are so effective for transactional systems.

Unstructured data is very different. Format such as images, videos, documents, application logs, and audio files does not naturally fit into rows and columns. The format is flexible, but that flexibility comes with different storage and retrieval changes. Instead of complex SQL queries, these files are typically stored in object storage systems and accessed through metadata or specialized indexing mechanisms.

In real-world architectures, you will almost always use both. An e-commerce platform might store customer and order information in a relational database, while product images, invoices, and user-generated content live in object storage. Understanding the nature of the data is the first step toward selecting the right storage technology, optimizing the performance, and designing systems that scale effectively.

Categories of Storage



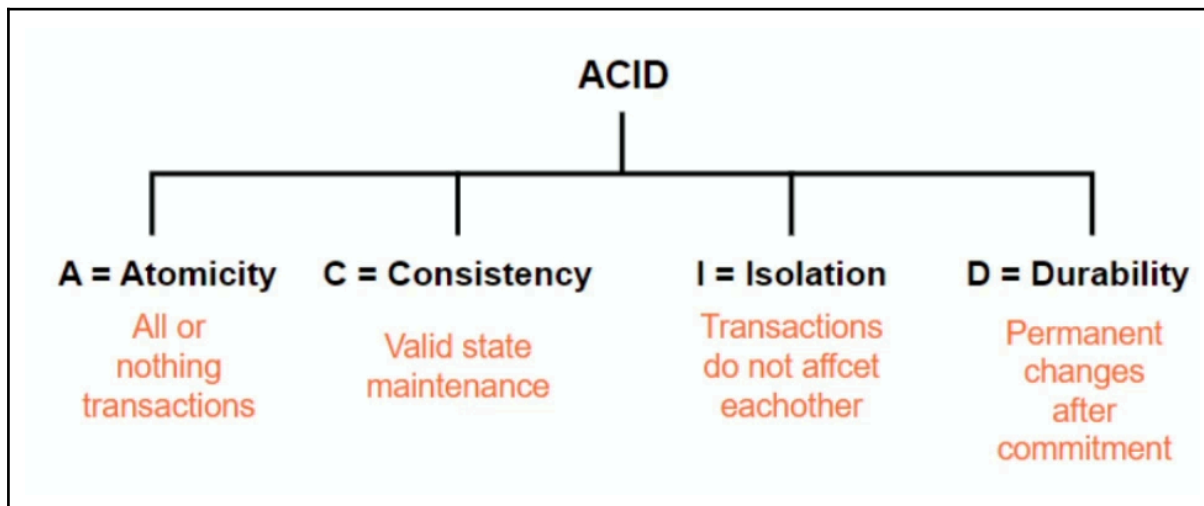
Now when architects talk about storage, they are not referring to a single technology. Modern systems use multiple storage categories, each designed for different access patterns, performance requirements, and scalability goals.

- **Database storage (SQL/NoSQL)** is optimized for managing application data that needs to be queried, updated and related to other data. Whether it is a relational database for transactions, or NoSQL databases for large-scale distributed workloads, databases are the backbone of most business applications.
- **Object storage (e.g., S3)** takes a different approach. Instead of storing data in tables or file systems, it stores objects that can scale to billions of items. This makes it a preferred choice for assets like images, videos, documents, backups, and log archives, where durability and massive scalability matter more than low-latency updates.
- **File storage (e.g., NFS)** provides the familiar folder and file hierarchy that operating systems and many legacy applications expect. It is useful when multiple servers or applications need shared access to the same files through a standardized file system interface.
- **Block storage (e.g., EBS)** is the lowest level of storage in which storage is presented as raw disk blocks, giving applications direct control over how data is organized. This delivers the high performance and low latency required by databases, VMs, and other I/O intensive workloads.

The key architectural lesson is that large-scale systems rarely rely on just one storage type. Most production systems combine databases, object storage, file storage, and block storage, using each where its strength provides the greatest value for their system.

Storage Properties

When evaluating a storage system, architects care less about where data is stored and more about the guarantees the storage system provides. These guarantees determine how the system behaves under failure, high traffic, and distributed deployment.



Durability is the first property. Once data is successfully written, it should survive crashes, hardware failures, and restarts. If a user uploads a document or completes a payment, losing that data is usually unacceptable. Achieving durability often requires techniques like replication, backups, and persistent disk storage.

Availability is next — the ability to access data whenever it's needed. Highly available systems continue serving requests even when individual servers, disks, or entire nodes fail. For customer-facing applications, availability often has a direct impact on business revenue.

Consistency focuses on data correctness. When a write occurs, subsequent reads should return the latest value rather than stale data. In distributed systems, maintaining strong consistency can be challenging because data may exist on multiple replicas across different locations.

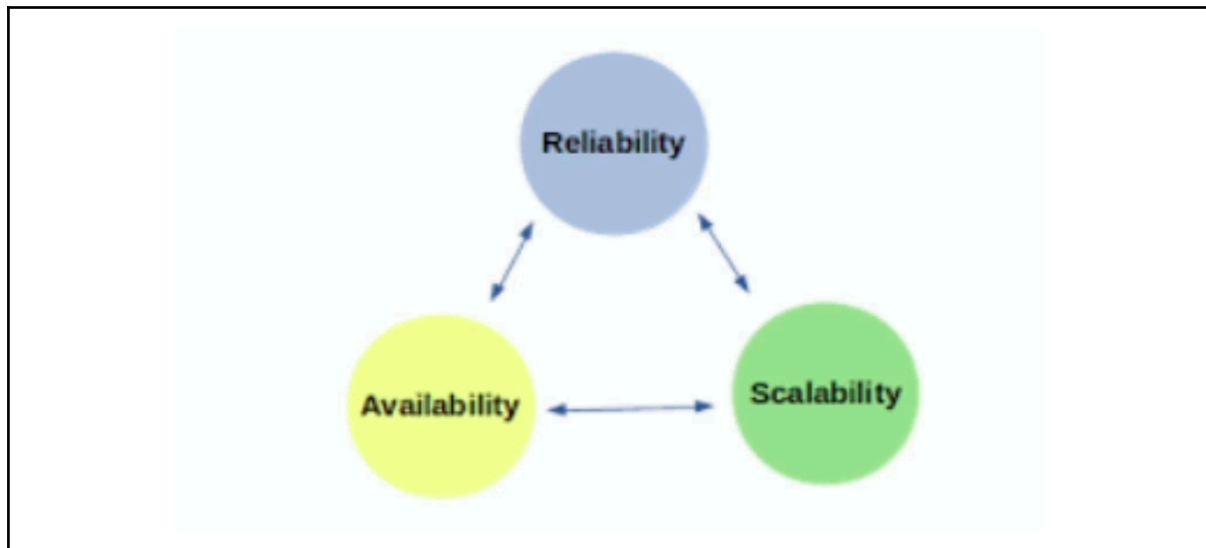
Isolation governs how concurrent operations interact. When multiple transactions run at the same time, isolation ensures that each one behaves as if it were running alone — one transaction shouldn't see the partial, uncommitted effects of another.

Atomicity is another important property, particularly in transactional systems. Some operations involve multiple changes that must succeed together — for example, transferring money between accounts requires both the debit and credit to complete as a single unit. If one fails, the entire operation must be rolled back to keep the system in a valid state.

As we will see throughout this lecture, many storage architecture decisions are ultimately trade-offs between durability, availability, consistency, performance, and cost. Understanding these properties is essential for making informed design choices at scale.

The Trade-offs in Storage Design

One of the most important lessons in system design is that storage decisions are rarely about finding the best solution. They are about finding the right balance. Every storage architecture involves trade-offs, and improving one characteristic often means sacrificing another one. At a high level, architects constantly balance 3 competing goals - scalability, reliability, and performance.



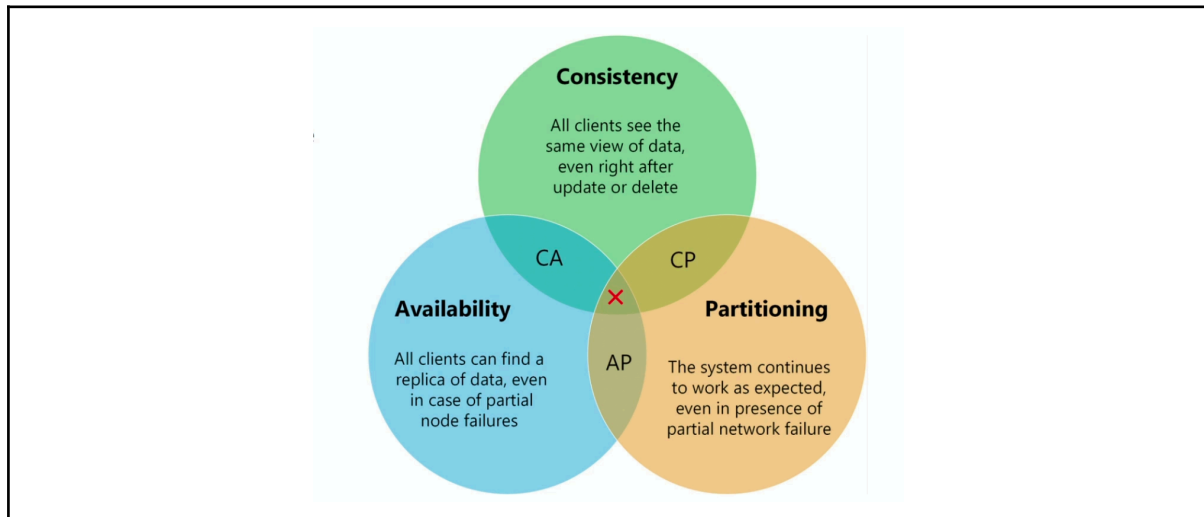
Scalability determines how well the system handles growth as data volumes, users, and traffic increases over time. Reliability focuses on keeping data safe and ensuring that the system continues operating despite hardware failures, network issues, or outages. And performance is all about delivering fast reads and writes with minimal latency.

Now the challenge is that these goals often pull in different directions. For example, replicating data across multiple servers improves reliability, but it can introduce additional latency and reduce performance. Optimizing for extremely low latency access might require keeping data close to the user, but that can make consistency and reliability harder to maintain at scale. This is why experienced architects don't ask what is the best storage solution. They ask what problem am I optimizing for. A banking system may prioritize reliability and consistency over raw performance, while a social media feed may prioritize scalability and responsiveness.

Ultimately, storage design is an exercise in making informed compromises. The value of a system designer lies not in eliminating the trade-off, but in choosing the ones that best support the business goals and the technical requirements of your system.

The CAP Theorem

The CAP theorem is one of the foundational ideas in distributed systems because it explains why storage systems make different architectural choices as they scale across multiple servers and regions. CAP tells us that when a distributed system experiences a network partition, meaning some nodes can no longer communicate reliably, you cannot simultaneously guarantee both perfect consistency and full availability. At that moment, a choice has to be made.



Let's look at the 3 properties involved.

- Consistency means every user sees the latest committed data, regardless of which node they are connected to.
- Availability means every request receives a response, even if some parts of the system are experiencing issues.
- Partition tolerance means that the system continues operating despite network failure between nodes.

The key insight is that partition tolerance is not really optional in large-scale distributed systems. Networks fail, links break, and regions become temporarily isolated. Since partitions are inevitable real-world systems are effectively choosing between consistency and availability during those failures. For example, a banking system may favor consistency, preferring to reject requests rather than to risk showing incorrect account balances. A social media feed, on the other hand, might favor availability, allowing users to continue interacting even if some data is temporarily stale.

A common interview mistake is thinking CAP is a permanent choice. It is not, as under normal conditions, many systems provide both consistency and availability. The trade-off only becomes visible when a network partition occurs. Understanding that distinction is very critical because CAP is ultimately about how a system behaves during failure, not when everything is working perfectly.

Types of Systems Based on CAP Trade-offs

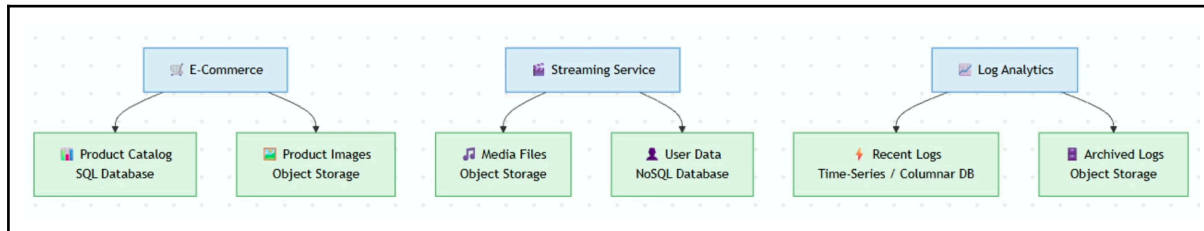
Once you understand the CAP Theorem, the next step is understanding real-world systems position themselves when those trade-offs become unavoidable. Different storage systems make different choices based on what they are optimizing for.

- **CP Systems (Consistency + Partition Tolerance):** These systems prioritize data correctness above everything else. During a network partition, they may refuse reads or writes rather than risk returning incorrect data. From a user's perspective, the system might appear temporarily unavailable. But the data you do receive is guaranteed to be correct. This approach is very common in banking, financial transactions, inventory management, and other domains where correct data can be more damaging than temporary downtime.
- **AP Systems (Availability + Partition Tolerance):** These systems choose to remain operational even when parts of the network cannot communicate. Users continue receiving responses, but some of those responses may contain stale data until replicas eventually synchronize. This trade-off works well for workloads like social feeds, product catalogs, recommendation systems, and content platforms where a slightly outdated response is usually acceptable.
- **CA Systems (Consistency + Availability):** In theory these systems provide both correct data and continuous availability. The catch is that this only works when network partitions never occur. Since partitions are unavoidable in distributed environments, a true CA System effectively exists only in single-node or tightly coupled deployments rather than large-scale distributed architectures.

The important lesson is not memorizing which database belongs to which category. It is understanding that the business impact of each choice there is. When designing a system, you are really answering a fundamental question. If the network fails, would you rather reject requests or serve potentially stale data? And this answer often determines whether a CP or AP approach is a better fit for your system.

Real-World Use Cases

The best way to understand storage architecture is to look at how real systems combine multiple storage technologies to solve different problems. In a production environment, it is very rare to find a system relying on a single storage type.



Ecommerce applications store their product information, inventory level, pricing and order in databases as they are highly structured and need efficient querying and transactional guarantees. At the same time, product images, videos, and downloadable assets are large unstructured objects that are better suited to object storage. Using the same storage solution for both would be insufficient and costly.

Streaming service follows a similar pattern. The media content itself, the movies, the songs, and the thumbnails, are stored in highly scalable object storage designed to handle massive files and global distribution. While user profiles, viewing history, recommendation, and session data are stored separately, often in NoSQL databases that are optimized for high volume reads and writes.

Log analytics is another interesting example. Modern applications generate enormous volumes of logs and telemetry data. Recent logs may be stored in specialized time series column databases to support fast searching and analytics, while older logs are moved to object storage for long-term retention at much lower cost.

The key architectural takeaway here is that storage selection should always follow the workload. Different data types have different access patterns, scalability requirements, and cost considerations. Successful systems embrace a polyglot storage approach, meaning using the right storage technology for the right job, rather than forcing everything into a single solution.

Summary & Key Takeaways

- Storage is foundational in every system
- Know your data type: structured vs. unstructured
- Choose storage based on properties, access patterns, and scale
- Real-world systems often use a combination of storage types
- What's next:
 - Understanding Database Models: SQL vs. NoSQL

Interview Question

- Why is storage a critical component in system design?
- How would you differentiate between structured and unstructured data?
- What are the different types of storage systems and their use cases?
- What do durability, availability, and consistency mean in the context of storage?
- What is atomicity, and where is it relevant?
- Can a system be both highly available and strongly consistent? Why or why not?
- What is the CAP theorem? Why is it important in distributed system design?
- Explain the difference between CP and AP systems with examples.
- Why is CA considered rare or impractical in distributed systems?
- How would you decide between consistency and availability when designing a real-world system?
- You're building a photo-sharing app. How would you design storage for photos vs. user metadata?
- What kind of storage system would you choose for an analytics pipeline handling logs and metrics?
- How does object storage differ from file and block storage in terms of access patterns and scalability?